01

Haskell

Часть 1. Основы языка

```
add :: Int -> Int -> Int
add x y = x + y

add 2 3  -- 5
```

Андреев Александр
Крапивина Ульяна
Савосина Дарья

История и версия языка

01 1987: комитет начинает разработку общего ленивого языка

02 1990: выходит Haskell 1.0

03 Haskell 98 и Haskell 2010: опубликованные стандарты

04 Имя языка связано с математиком Хаскеллом Карри

```
-- Стандарт языка: Haskell 2010
-- Реализация: GHC
-- Примеры: без расширений

ghc -XHaskell2010 Hello.hs
```

Стандарт языка и версия компилятора — разные вещи

Как устроено программирование на Haskell

01 Функции вычисляют результат из аргументов

02 Типы проверяются до выполнения программы

03 Связывания имён неизменяемы

04 Выражения вычисляются по мере необходимости

```
square :: Int -> Int
square x = x * x

square 4     -- 16
square 4 + 1 -- 17
```

Побочные эффекты явно отражаются в типе IO

Окружение и запуск

01 GHCup устанавливает инструменты Haskell

02 GHC компилирует исходники в программу

03 GHCi позволяет пробовать выражения

04 runghc запускает файл без отдельного шага сборки

```
ghc --version
ghci --version

cd src
runghc -XHaskell2010 Hello.hs
```

Установка для своей ОС: haskell.org/ghcup

Первая программа

01 main — точка входа программы

02 IO () — действие, которое не возвращает полезного значения

03 putStrLn выводит строку и перевод строки

04 Сигнатура типа полезна, но может быть выведена

```haskell
module Main where

main :: IO ()
main = putStrLn "Hello, Haskell!"
```

Вывод

```
Hello, Haskell!
```

Файл: Hello.hs

GHCi: быстрый эксперимент

01 :load загружает исходный файл

02 :type показывает тип выражения

03 :reload перечитывает изменения

04 :quit завершает сеанс

```
ghci -XHaskell2010 Basics.hs

:type add
-- add :: Int -> Int -> Int
add 2 3
-- 5
:quit
```

Первая строка — терминал, остальные — команды GHCi

Базовые типы

01 Int: целое с ограниченным диапазоном

02 Integer: целое произвольной точности

03 Double: число с плавающей точкой

04 Bool, Char и String: логика, символ и строка

```
42         :: Int
10 ^ 30    :: Integer
3.14       :: Double
True       :: Bool

'H'        :: Char
"Haskell" :: String
```

String — синоним [Char], то есть списка символов

Операции над типами

01 data создаёт собственный тип и конструкторы

02 type задаёт синоним существующего типа

03 newtype создаёт отдельный тип-обёртку

04 Образцы позволяют разобрать значение

```
type Name = String
newtype Age = Age Int

data Shape = Circle Double
           | Rectangle Double Double

area :: Shape -> Double
area (Circle r) = pi * r * r
area (Rectangle w h) = w * h
```

Синоним Name не создаёт новый тип, а Age создаёт

Списки и кортежи

01 Список содержит элементы одного типа

02 Кортеж объединяет фиксированное число значений

03 [] — пустой список, : добавляет элемент в начало

04 ++ соединяет списки

```
[1, 2, 3] :: [Int]
("Ada", 21) :: (String, Int)

0 : [1, 2]    -- [0,1,2]
[1,2] ++ [3]  -- [1,2,3]
[1..5]        -- [1,2,3,4,5]
```

[1, True] не проходит проверку типов

Операции и преобразования типов

01 + и * работают с числовыми значениями

02 div выполняет целочисленное деление

03 / используется для дробного деления

04 fromIntegral явно преобразует целое в числовой тип

```
7 `div` 2  -- 3
7 / 2      -- 3.5

half :: Int -> Double
half n = fromIntegral n / 2

half 7     -- 3.5
```

Обратные кавычки позволяют вызвать функцию между аргументами

Имена и области видимости

01 = связывает имя с выражением

02 let ... in создаёт локальные связывания

03 Внутреннее имя может затенить внешнее

04 Внешнее связывание при этом не изменяется

```
outer :: Int
outer = 10

scoped :: Int
scoped = let outer = 3
         in outer + 1

-- scoped = 4, outer = 10
```

Связывание имени и изменение значения — разные операции

Владение и передача данных

01 Для обычных значений нет обязательного move

02 Аргумент можно использовать повторно

03 Неизменяемые структуры могут разделяться

04 Память освобождает сборщик мусора

```
xs :: [Int]
xs = [1,2,3]

ys :: [Int]
ys = 0 : xs

-- xs по-прежнему доступен
-- xs = [1,2,3]
-- ys = [0,1,2,3]
```

Изменяемое состояние задают отдельно, например через IORef или ST

Функции: аргументы и результат

01 Сигнатура описывает входные и выходные типы

02 Аргументы разделяются пробелами

03 Последний тип в записи — результат

04 Скобки группируют выражения

```
add :: Int -> Int -> Int
add x y = x + y

add 2 3          -- 5
square (add 2 3) -- 25

-- square определена в Basics.hs
```

Тип результата здесь Int

Входные данные и результат функции

01 Аргументы задают входные значения

02 Функция возвращает одно значение

03 Кортеж объединяет несколько компонентов

04 Тип результата может отличаться от типа входа

```
step :: (Int, Int) -> (Int, Int)
step (x, y) = (x + 1, y - 1)

step (2, 3)  -- (3,2)

isZero :: Int -> Bool
isZero x = x == 0
```

Пара значений тоже является одним значением кортежного типа

Каррирование и частичное применение

01 Int -> Int -> Int читается как Int -> (Int -> Int)

02 После первого аргумента остаётся функция

03 Частичное применение создаёт новую функцию

04 Это удобно для повторяющихся преобразований

```
add :: Int -> Int -> Int
add x y = x + y

addTen :: Int -> Int
addTen = add 10

addTen 5  -- 15
```

add 10 ожидает ещё один аргумент

Условия: if и guards

01 if — выражение, которое возвращает значение

02 Обе ветви if должны иметь один тип

03 Guards проверяются сверху вниз

04 otherwise равен True и задаёт последний случай

```
absolute :: Int -> Int
absolute x = if x < 0 then -x else x

signLabel :: Int -> String
signLabel x
  | x < 0     = "negative"
  | x == 0    = "zero"
  | otherwise = "positive"
```

Логические операции: &&, ||, not

Сопоставление с образцом

01 Образец описывает форму входного значения

02 [] означает пустой список

03 x:xs выделяет первый элемент и остаток

04 _ обозначает часть, которая не нужна

```
describe :: [Int] -> String
describe []     = "empty"
describe [_]    = "one item"
describe (_:_)  = "several items"

-- Порядок вариантов важен
```

Последний образец подходит и к одноэлементному списку

Рекурсия на списке

01 Базовый случай завершает вычисление

02 Шаг обрабатывает один элемент

03 Рекурсивный вызов получает меньший список

04 Здесь рассматриваем конечные списки

```
sumList :: [Int] -> Int
sumList []     = 0
sumList (x:xs) = x + sumList xs

sumList [2,3,4]
-- 2 + (3 + (4 + 0))
-- 9
```

Оба образца вместе покрывают все формы списка

Циклы и обработка списков

01 В чистом коде повторение выражают рекурсией

02 map применяет функцию к элементам

03 filter отбирает элементы по условию

04 Функции можно передавать как аргументы

```
map square [1,2,3]
-- [1,4,9]

filter even [1..6]
-- [2,4,6]

map square (filter even [1..6])
-- [4,16,36]
```

Вместо for часто используют рекурсию и функции высшего порядка

Итерация и повторение действий

01 Списки можно обходить лениво

02 take ограничивает число запрошенных элементов

03 forM_ повторяет действия IO для элементов

04 Результаты действий при таком обходе отбрасываются

```
import Control.Monad (forM_)

main :: IO ()
main = forM_ [1..3] print

-- Вывод:
-- 1
-- 2
-- 3
```

forM_ — библиотечная функция, а не отдельный оператор цикла

Свёртка: список в одно значение

01 foldr задаёт шаг и начальное значение

02 Для суммы шаг — (+), начало — 0

03 Для произведения шаг — (*), начало — 1

04 Свёртка обобщает знакомую рекурсию

```
foldr (+) 0 [2,3,4]
-- 2 + (3 + (4 + 0))
-- 9

foldr (*) 1 [2,3,4]
-- 24
```

foldr :: (a -> b -> b) -> b -> [a] -> b

Лямбда-функции и замыкания

01 Лямбда задаёт функцию без отдельного имени

02 \x -> ... описывает параметр и тело

03 Замыкание сохраняет доступ к внешнему окружению

04 makeAdder создаёт функцию с выбранным n

```
map (\x -> x * 2) [1,2,3]
-- [2,4,6]

makeAdder :: Int -> (Int -> Int)
makeAdder n = \x -> n + x

makeAdder 10 5  -- 15
```

Полученная функция использует n из внешней области

Ленивые вычисления

01 Результат вычисляется настолько, насколько нужен

02 Список [1..] описывает бесконечную последовательность

03 take запрашивает только первые элементы

04 Попытка вывести весь список не завершится

```
naturals :: [Integer]
naturals = [1..]

take 5 naturals
-- [1,2,3,4,5]

take 3 (map square [1..])
-- [1,4,9]
```

Ленивость не гарантирует ускорение или уменьшение памяти

Локальные определения и отступы

01 where добавляет определения к функции

02 let ... in создаёт отдельное выражение

03 Отступы задают группировку объявлений

04 do объединяет действия IO

```
circleArea :: Double -> Double
circleArea r = pi * squared
  where
    squared = r * r

helloTwice :: IO ()
helloTwice = do
  putStrLn "Hello"
  putStrLn "Haskell"
```

Для отступов используйте пробелы последовательно

Основные конструкции первой части

01 Неизменяемые связывания и лексическая область

02 Типы аргументов и результата функции

03 Рекурсия, замыкания и частичное применение

04 Условия, образцы и обход коллекций

```
-- Проверьте себя
map (add 10) [1,2,3]

-- Результат?
-- Тип результата?
-- Почему add 10 — функция?
```

Тип выражения map (add 10) [1,2,3] — [Int]

Источники и примеры

01 Haskell 2010 Language Report

02 GHC User’s Guide

03 GHCup: установка инструментов

04 Документация стандартного пакета base

```
haskell.org/onlinereport/haskell2010/
downloads.haskell.org/ghc/9.0.1/
www.haskell.org/ghcup/

Стандарт: Haskell 2010
Реализация: GHC
```

Источники описывают язык, реализацию и стандартные функции